Day 1 – What is the Web? | Install VS Code | Create First HTML Page

1. What is the Web?

The **Web** (World Wide Web) is a global system of information accessed through the internet. It allows users to browse websites and interact with content using browsers like Chrome, Firefox, etc.

Example:

When you visit Google, YouTube, or Instagram — you're using the web.
Each website has a unique address (called a URL), such as:
https://www.google.com

2. What is a Website?

A **website** is a group of related **web pages** that live under one domain name. These pages contain text, images, videos, buttons, and more.

Websites are built using **HTML, CSS, and JavaScript**.

3. How the Web Works (Basic Understanding):

1. You type a web address in your browser.
2. The browser sends a request to the server.
3. The server sends back the webpage.
4. The browser displays the page to you.

4. Tools Required for Web Development

To begin your journey in web development, you'll need:

- 💻 **VS Code** (Text Editor for writing code)
- 🌐 **Browser** (To test your website)

5. Installing VS Code

🔧 Follow these steps:

1. Visit: https://code.visualstudio.com
2. Click on **Download for Windows** (or your system's OS)
3. Install it like any regular software
4. Open **Visual Studio Code**

6. Create Your First HTML Page

📝 Step-by-Step:

✅*Step 1: Create a New File*

- In VS Code → Go to File > New File
- Save the file as: index.html

✅*Step 2: Write HTML Code*

```
<!DOCTYPE html>
<html>
 <head>
  <title>My First Web Page</title>
 </head>
 <body>
  <h1>Hello, I am Dev Darji</h1>
  <p>This is my first HTML page in the internship.</p>
 </body>
</html>
```

✅*Step 3: Run the File*

- Right-click on the file → Select **Open with Live Server** (if extension is installed)
- OR open the file manually in any browser

7. Understanding the HTML Tags

Tag	Purpose	Description
`<html>`	HTML document	The root of your webpage
`<head>`	Page metadata	Contains title and links
`<title>`	Page title	Shows in browser tab
`<body>`	Content section	All visible content
`<h1>`	Heading 1	Largest heading
`<p>`	Paragraph	Normal text block

IMPORTANT NOTES for All Students:

- ✅ **Submit your Day 1 task by email** to:
  📧 **workdinarventure@gmail.com**
- ✅ The code and explanation provided above is just for **reference**.
- 🔍 You are expected to do a **bit of your own research on Google** and try to **understand the concept clearly**.
- 🚫 **Do not directly copy-paste** my code. Try to make your code a little **unique, attractive, and different** in your own creative way.

Day 1 Task:

1. Install **VS Code**
2. Create an `index.html` file
3. Write your own version of the sample HTML code
4. Open it in your browser
5. Take a screenshot of your **code and the output**
6. Submit your task via email at: 📧 workdinarventure@gmail.com

Day 2 – HTML: Headings, Paragraphs, Links, Images

1. What is HTML?

HTML stands for **HyperText Markup Language**. It is the standard language used to create **webpages** and give them structure.

◆ 2. Headings in HTML

HTML provides 6 levels of headings from `<h1>` to `<h6>`.
`<h1>` is the largest and most important; `<h6>` is the smallest.

🛠 Example:

```
<h1>This is Heading 1</h1>
<h2>This is Heading 2</h2>
<h3>This is Heading 3</h3>
<h4>This is Heading 4</h4>
<h5>This is Heading 5</h5>
<h6>This is Heading 6</h6>
```

3. Paragraphs in HTML

Paragraphs are written using the `<p>` tag. It defines blocks of text.

🛠 Example:

```
<p>This is my first paragraph. HTML makes content easy to display on web pages.</p>
```

4. Links in HTML

Links are created using the <a> tag. It is used to navigate to another page or website.

🛠️ Syntax:

Visit Google

5. Images in HTML

Images are inserted using the <img> tag. It's a self-closing tag.

🛠️ Syntax:

****

Attributes:

- src → URL or path of the image
- alt → Alternative text (for SEO and accessibility)
- width / height → To set image size

Day 2 Task Checklist:

1. Create a new file called `day2.html`
2. Add headings from `<h1>` to `<h6>`
3. Write at least 1 paragraph
4. Add one clickable link
5. Add at least one image (from internet or local)
6. Open the file in a browser and take a screenshot of both code and output
7. Submit it via email to: workdinarventure@gmail.com

Sample Code – Combine All:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Day 2 – HTML Basics</title>
  </head>
  <body>
    <h1>Welcome to Web Development</h1>
    <p>This is my paragraph written using the p tag.</p>
    <h2>Link Example</h2>
    <a href="https://www.wikipedia.org" target="_blank">Visit Wikipedia</a>

    <h2>Image Example</h2>
    <img src="https://via.placeholder.com/300" alt="Demo Image" width="300" />
  </body>
</html>
```

IMPORTANT NOTES for Students:

- ✓ You must **submit your Day 2 task** to the email below:
  workdinarventure@gmail.com

- 🔍 Use this content only as a **reference**.
- ❌ **Do not directly copy-paste**.
  ✅ Instead, understand the tags and **create your own HTML page**.
- 💡 Try adding your **own headings, links, and images** for creativity.
- 🔎 Google and explore a little more about these tags to expand your knowledge.

Day 3 – HTML: Lists, Tables, and Forms

---

1. HTML Lists

Lists help display items in **a structured format**. There are 3 types of lists in HTML:

✅ Ordered List `<ol>`

Shows items in numbered format.

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>
```

✅ Unordered List `<ul>`

Shows items with bullet points.

```
html
CopyEdit
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Orange</li>
</ul>
```

✅ Description List `<dl>`

Used for definitions or descriptions.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

---

2. HTML Tables

Tables are used to display data in **rows and columns**.

🛠️ Example:

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>City</th>
  </tr>
  <tr>
    <td>Dev Darji</td>
    <td>22</td>
    <td>Ahmedabad</td>
  </tr>
  <tr>
    <td>Riya</td>
    <td>21</td>
    <td>Surat</td>
  </tr>
</table>
```

✅ Tags Used:

- <table> → defines the table
- <tr> → table row
- <th> → table heading
- <td> → table data cell

---

3. HTML Forms

Forms are used to **collect user input**.

🛠️ Example:

```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" /><br><br>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" /><br><br>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" /><br><br>

  <input type="submit" value="Register" />
</form>
```

✅ Common Input Types:

- text – for normal input
- email – for email addresses
- password – for password fields
- submit – submit the form

📌 IMPORTANT NOTES for Students:

- ✅ Submit your Day 3 task to:
  📧 **workdinarventure@gmail.com**
- 🔍 Use this code **only as reference**.
  Do your own research on Google and try to modify or improve the design.
- 🎯 Avoid simple copy-pasting — **add your own creativity** to the page.

Day 3 Task Checklist:

1. Create a file called day3.html
2. Create:
   - One **Ordered List**
   - One **Unordered List**
   - One **Description List**
3. Create a **Table** with at least 3 rows and 3 columns
4. Create a **Form** with name, email, and password fields + submit button
5. Open in browser → take a screenshot of both code and output
6. Submit to: 📧 **workdinarventure@gmail.com**

Day 4 – HTML Mini Project: “Personal Info Form”

◆ Objective

Today, you will **build a mini HTML form-based project** where users can input their personal information such as name, contact details, and preferences. This task will strengthen your understanding of HTML form elements.

🎯 Project Title: Personal Info Form

What You’ll Learn:

- How to build a structured form
- How to use input types
- How to group form elements
- How to use labels effectively

important Instructions for Students:

- ✅ This project must be submitted to:
  📧 **workdinarventure@gmail.com**
- Reference : online website research and create task
- Try to improve the form by:
- Adding your own styling or layout
- Including more input types or validations
- Explore more on Google and YouTube to learn beyond this task.

Day 5 – CSS: Selectors, Colors, Fonts, and Backgrounds

🎯 Objective:

Today you will learn the **basics of CSS styling** and how to apply different selectors, set colors, use fonts, and apply backgrounds to elements.

What is CSS?

CSS (Cascading Style Sheets) is used to **style HTML elements**. It controls how the webpage looks — layout, colors, fonts, spacing, etc.

---

1. CSS Selectors

Selectors decide **which HTML elements** the CSS will style.

Examples:

```
/* Element Selector */
p {
  color: blue;
}

/* ID Selector */
#heading {
  font-size: 24px;
}

/* Class Selector */
.card {
  border: 1px solid gray;
  padding: 10px;
}
```

---

2. Colors in CSS

You can use colors in different formats:

◆ Examples:

```
body {
  background-color: #f0f0f0;
}

h1 {
  color: red;
}

p {
  color: rgb(0, 128, 255);
}
```

---

3. Fonts in CSS

Font Styling Example:

```
body {
  font-family: 'Arial', sans-serif;
  font-size: 16px;
  font-weight: bold;
  font-style: italic;
}
```

---

4. Backgrounds in CSS

You can set **background colors, images, and gradients**.

Background Color:

```
div {
  background-color: lightblue;
}
```

Background Image:

```
body {
  background-image: url("background.jpg");
  background-size: cover;
  background-repeat: no-repeat;
}
```

📌 IMPORTANT NOTE FOR STUDENTS:

- 📧 Submit your work to:workdinarventure@gmail.com
- 🔍 This task is for **learning purpose** — use the code above only for reference.
- □ Please **do your own research** and add creativity to your CSS.
- ✨ Try different colors, fonts, and backgrounds on your own.

✅ Day 5 Task Checklist:

1. Create a file named `style_practice.html` and `style.css`
2. Create a basic HTML structure with:
    - A heading, a paragraph, and a container `<div>`
3. Style it using:
    - **Element selector** for the paragraph
    - **ID selector** for the heading
    - **Class selector** for the div
4. Apply:
    - Font style and font size
    - Text color and background color
    - Optional: Add background image
5. Open in browser → take screenshot of output and code
6. Submit it to: 📧 **workdinarventure@gmail.com**

Day 6 – CSS Box Model, Margin, Padding, and Border

Objective:

Today, you'll learn about the **Box Model in CSS**, which helps control the **spacing and layout** of elements on a webpage. Understanding this is essential for designing neat and responsive layouts.

What is the Box Model?

Every HTML element is considered a **box** in CSS. The **box model** consists of:

```
| Margin |
| Border |
| Padding |
| Content |
```

---

1. Content

The actual text or image inside the box.

2. Padding

Space **inside** the box, between the content and the border.

```
p {
  padding: 20px;
}
```

3. Border

The line surrounding the element (outside padding).

```
p {
  border: 2px solid black;
}
```

4. Margin

Space **outside** the border. It pushes the element away from other elements.

```
p {
  margin: 30px;
}
```

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Box Model</title>
  <style>
    .box {
      width: 300px;
      padding: 20px;
      border: 2px solid blue;
      margin: 30px;
      background-color: lightyellow;
    }
  </style>
</head>
<body>
  <h1>CSS Box Model Demo</h1>
  <div class="box">
    This is a box with padding, border, and margin.
  </div>
</body>
</html>
```

Your Task (Day 6):

🛠️ Create a new file named `boxmodel.html` and follow the steps:

✅ Task Steps:

1. Create a `<div>` with class `box`
2. Set width: `300px`, background-color: `any light color`
3. Apply:
    - Padding: `20px`
    - Border: `2px solid color of your choice`
    - Margin: `30px`
4. Add text inside the box
5. Try changing each value to observe how it affects the layout

📌 Important Instructions for Students:

- 📧 Submit your task at: **workdinarventure@gmail.com**
- ❌ Do not just copy the example code
- ✅ Use this as **reference only**, and make your design unique
- 🔍 Search online or on YouTube if needed
- 💡 Try to explore box shadows or `border-radius` as extra practice!

Day 7 – CSS Flexbox + Layout Practice

Objective:

Today, you'll learn **Flexbox**, a modern CSS layout system used to create responsive layouts easily. By the end of this session, you'll be able to align, space, and arrange elements in rows or columns.

What is Flexbox?

Flexbox (Flexible Box) is a CSS layout module that makes it easier to design flexible and responsive layout structures without using float or positioning.

---

1. Display: Flex

To use Flexbox, you need to set the container's display to `flex`.

```
.container {
  display: flex;
}
```

---

2. Main Flexbox Properties

◆ **flex-direction:**

Defines the direction of items.

```
flex-direction: row;        /* default */
flex-direction: column;
```

---

justify-content:

Aligns items **horizontally** (main axis).

```
justify-content: flex-start;  /* default */
justify-content: center;
justify-content: space-between;
justify-content: space-around;
```

`align-items`:

Aligns items **vertically** (cross axis).

```
align-items: stretch;       /* default */

align-items: center;
align-items: flex-start;
align-items: flex-end;
```

`flex-wrap`:

Controls whether items stay in one line or wrap.

```
flex-wrap: wrap;
```

Example: Simple Flex Layout

```
<!DOCTYPE html>
<html>
<head>
  <title>Flexbox Layout</title>
  <style>
    .container {
      display: flex;
      justify-content: space-around;
      align-items: center;
      flex-wrap: wrap;
      height: 200px;
      background-color: #f0f0f0;
    }

    .box {
      width: 100px;
      height: 100px;
      background-color: #4caf50;
      color: white;
      text-align: center;
      line-height: 100px;
      margin: 10px;
      font-weight: bold;
```

```
    }
  </style>
</head>
<body>
  <h2>CSS Flexbox Example</h2>
  <div class="container">
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
    <div class="box">Box 3</div>
    <div class="box">Box 4</div>
  </div>
</body>
</html>
```

Day 7 Task: Layout Practice Using Flexbox

Task Steps:

1. Create a new file named `flexbox_layout.html`
2. Create a `.container` and at least 4 `.box` inside it
3. Use Flexbox to:
    - Arrange items horizontally
    - Use `justify-content` and `align-items`
    - Try `flex-wrap: wrap` with smaller screen
4. Make each box a different color
5. Add text inside each box
6. Try responsiveness by resizing the browser window

Important Instructions for Students:

- 📧 Submit your task at: **workdinarventure@gmail.com**
- ❌ Don't directly copy the above code
- 💡 This is just for reference. Try your own unique layout
- 🔍 Google and learn more Flexbox properties
- 🎨 Make your layout more attractive

Day 8 – Web Project: “Basic Portfolio Page”

Objective:

Today, you will **apply everything you've learned** in HTML and CSS to create a **Basic Portfolio Web Page**. This is your **first mini-project** to showcase your own information using real web structure and styling.

What to Include in the Portfolio:

Basic Sections:

1. **Header** – Your name and profession
2. **About Me** – Short paragraph about yourself
3. **Skills** – List of your technical or soft skills
4. **Projects** – Add 2–3 sample project titles with short descriptions
5. **Contact** – Simple contact form (Name, Email, Message)
6. **Footer** – Copyright and basic credits

Tools You’ll Use:

- **HTML**: To structure the page
- **CSS**: To design the page (Fonts, Colors, Layouts, Spacing)
- **Flexbox**: For layout and alignment

Styling Ideas (in **style.css**):

- Use Flexbox to organize sections
- Add hover effects on buttons or links
- Use Google Fonts for better typography
- Add a light color scheme (or dark mode!)

Day 8 Task Checklist:

1. Create folder: portfolio_project/
2. Create files: index.html, style.css
3. Build all sections (header, about, skills, projects, contact, footer)
4. Style everything using Flexbox, padding, margin, and fonts
5. Add creative touches: background colors, borders, hover effects
6. Test in browser and take a screenshot
7. Zip your folder (portfolio_project.zip)

📌 Submission Instructions:

- 📧 Submit your ZIP file to: **workdinarventure@gmail.com**
- 📂 Subject: Day 8 - Basic Portfolio Project Submission
- 🔍 Research your own style, layout, and add personal creativity

Day 9 – JavaScript: Variables, Data Types, and Operators

Objective:

Today, you'll begin with the basics of **JavaScript** — the programming language of the web. You'll understand how to **declare variables**, work with **data types**, and use **operators** to build logic.

---

1. What is JavaScript?

JavaScript is a **programming language** used to create interactive effects inside web browsers. It can **change content**, **validate forms**, **animate elements**, and more.

How to Link JavaScript to HTML

```
<script src="script.js"></script>
```

Or write directly inside:

```
<script>
  console.log("Hello from JS");
</script>
```

---

2. Variables in JavaScript

Variables store data values. You can declare variables using:

```
var name = "Dev";       // old way (avoid)
let age = 25;           // preferred
const pi = 3.14;        // constant value
```

---

3. Data Types

JavaScript supports different types of data:

Type	Example
String	`"Hello"`
Number	`42, 3.14`
Boolean	`true, false`
Array	`[1, 2, 3]`
Object	`{name: "Dev"}`
Undefined	`let x;`
Null	`let y = null;`

---

4. Operators

➤ Arithmetic Operators:

```
+   // Addition
-   // Subtraction
*   // Multiplication
/   // Division
%   // Modulus (remainder)
```

➤ Assignment Operators:

```
=     // assign
+=    // x += 5 → x = x + 5
-=    // x -= 2
```

➤ Comparison Operators:

```
==    // equal to (value)
===   // equal to (value & type)
!=    // not equal
>     // greater than
<     // less than
>=    // greater than or equal to
```

➤ Logical Operators:

```
&&   // AND
||   // OR
!    // NOT
```

Example: Try in Console or Script

```
let name = "Dev";
let age = 22;
let isStudent = true;

console.log("Name:", name);
console.log("Age + 5 =", age + 5);
console.log("Is student?", isStudent);

let x = 10;
x += 5;
console.log("x =", x);

console.log(10 > 5);  // true
```

Day 9 Task

Task Steps:

1. Create a file named `script.js`
2. Declare variables for:
    - Your name, age, city
    - A boolean for student or not
3. Perform arithmetic on numbers (add, subtract)
4. Use comparison operators to check your age is greater than 18
5. Log all values in the browser console

📌Submission Instructions:

- 📧 Submit your file to: **workdinarventure@gmail.com**
- 📂 Subject: `Day 9 - JavaScript Basics Task`
- ❗ Don't just copy examples
- 🔍 Do small research and customize output
- 🧠 Focus on writing clean and readable code

Day 10 – JavaScript: Functions, If-Else, and Loops

Objective:

Learn how to write **functions**, use **conditions (if-else)** to make decisions, and repeat actions using **loops** in JavaScript.

---

1. JavaScript Functions

Functions are **blocks of code** designed to perform a task. You can **call** them whenever needed.

Function Syntax:

```
function greet() {
  console.log("Hello, student!");
}

greet(); // Calling the function
```

Function with Parameters:

```
function greetUser(name) {
  console.log("Hello, " + name + "!");
}

greetUser("Dev");
```

---

2. If-Else Conditions

Used to perform different actions based on **conditions**.

Syntax:

```
let age = 20;

if (age >= 18) {
  console.log("You are an adult.");
} else {
  console.log("You are a minor.");}
```

Else If Example:

```
let marks = 85;

if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 75) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

---

3. JavaScript Loops

Loops are used to **repeat a block of code** multiple times.

◆ For Loop:

```
js
CopyEdit
for (let i = 1; i <= 5; i++) {
  console.log("Count:", i);
}
```

◆ While Loop:

```
let i = 1;
while (i <= 3) {
  console.log("While loop:", i);
  i++;
}
```

Why Use Functions & Loops?

- **Functions**: Avoid repeating code. Reusable.
- **Loops**: Automate repetitive tasks.
- **If-Else**: Make decisions based on data/conditions.

Practice Task:

1. Create a file: `day10.js`
2. Write a function `checkAge(age)`:
    - If age ≥ 18 → print "Eligible to vote"
    - Else → print "Not eligible"
3. Write a function `tableOf(n)` to print the multiplication table of any number.
4. Use `for` loop to print numbers 1 to 10
5. Use `while` loop to print odd numbers from 1 to 9
6. Use `if-else` inside loop to check if each number (1–10) is even or odd.

📌 Submission Instructions:

- 📧 Email your file to: **workdinarventure@gmail.com**
- 📂 Subject: `Day 10 - JavaScript Functions Task`
- ❗ Don't copy code directly — use it only for reference.
- 🔍 Google and experiment with different logic
- ✨ Make your code clean, creative, and customized.

Day 11 – JavaScript: Events and DOM Manipulation

Objective:

Learn how to handle **user interactions (events)** and change or control the webpage using **DOM (Document Object Model) Manipulation**.

---

1. What is the DOM?

The **DOM (Document Object Model)** is a programming interface for HTML. It allows JavaScript to **access and change the content, structure, and style** of your webpage.

Example:

```
<p id="myPara">Hello!</p>
```

JavaScript can access this paragraph using:

```
let para = document.getElementById("myPara");
para.innerText = "Hi, I changed!";
```

---

2. Accessing HTML Elements

Method	Use
`getElementById()`	Select element by ID
`getElementsByClassName()`	Select elements by class
`getElementsByTagName()`	Select elements by tag
`querySelector()`	Select using CSS selector

Example:

```
let title = document.querySelector("h1");
title.style.color = "blue";
```

---

3. JavaScript Events

Events are actions that happen on a webpage (like clicks, hover, keypress, etc.)
You can write code that **runs when an event occurs**.

◆ Common Events:

Event	Triggered When...
`onclick`	Element is clicked
`onmouseover`	Mouse hovers over
`onkeyup`	Key is released
`onsubmit`	Form is submitted

---

4. Event Listener (Recommended Way)

```
document.getElementById("btn").addEventListener("click", function() {
  alert("Button was clicked!");
});
```

Example with DOM Manipulation:

```
<button id="btn">Click Me</button>
<p id="text">Original Text</p>

<script>
  let btn = document.getElementById("btn");
  let text = document.getElementById("text");

  btn.addEventListener("click", function() {
    text.innerText = "Text changed!";
    text.style.color = "green";
  });
</script>
```

Practice Task:

1. Create a file: `day11.html`
2. Add a heading `<h2>` and a button
3. On button click:
    - Change the heading text
    - Change the color of the text
4. Create a form with a name input field and submit button
5. On form submit:
    - Prevent page reload using `event.preventDefault()`
    - Show alert with name input value
6. Add mouseover event on a paragraph to change its background

---

📌 Submission Instructions:

- 📧 Send your file to: **workdinarventure@gmail.com**
- 📂 Subject: `Day 11 - Events and DOM Manipulation`
- ❗ Use this code as reference only
- 🔍 Google and experiment for better results
- 🚀 Make your interaction smooth and user-friendly

Day 12 – JavaScript Mini Project: Simple Calculator

Objective:

Create a **Simple Calculator** using HTML, CSS, and JavaScript that can perform **basic arithmetic operations** like addition, subtraction, multiplication, and division.

---

1. Project Overview

You'll build a calculator that includes:

- A display screen
- Buttons for numbers (0–9)
- Buttons for operations (+, –, ×, ÷)
- A clear (`C`) and equals (=) button
- Logic for calculations using JavaScript

---

2. Technologies Used

- HTML (for structure)
- CSS (for design)
- JavaScript (for logic/functionality)

📌 Submission Instructions:

- 📧 Email your complete project (HTML, CSS, JS files) to: **workdinarventure@gmail.com**
- 📁 Subject: `Day 12 - Simple Calculator Project`
- 🔍 Add your own **creative design**, button styles, and background
- Don't copy-paste; understand, rewrite, and explore on your own

Day 13 – Web Hosting: Use GitHub Pages

Objective:

Learn how to **host your static website for free** using **GitHub Pages** so your projects are accessible online.

---

1. What is GitHub Pages?

- GitHub Pages is a **free service** to host **static websites** (HTML, CSS, JS) directly from your GitHub repository.
- Perfect for portfolios, projects, or small websites.
- Your site will be live on the web with a URL like `https://username.github.io/repository-name/`

---

2. Steps to Host Website on GitHub Pages

Step 1: Create a GitHub Account

- Go to github.com and sign up if you don't have an account.

Step 2: Create a New Repository

- Click **New** repository
- Give it a name (e.g., `my-portfolio`)
- Choose **Public** repository
- Check **Add a README file** (optional)
- Click **Create repository**

Step 3: Upload Your Website Files

- On the repo page, click **Add file** > **Upload files**
- Upload your website files (`index.html`, CSS, JS, images, etc.)
- Commit the changes

Step 4: Enable GitHub Pages

- Go to **Settings** tab in your repo
- Scroll to **Pages** section (left sidebar)
- Under **Source**, select branch `main` (or `master`) and folder `/root`
- Click **Save**

Step 5: Access Your Live Website

- After few seconds, GitHub will provide a URL like:
  `https://yourusername.github.io/repository-name/`
- Open the URL in your browser — your website is live!

---

3. Important Tips

- Your main HTML file **must be named** `index.html` for GitHub Pages to work properly.
- Changes pushed to the repo automatically update the live site.
- Use GitHub Desktop or Git commands for easier file management (optional, advanced).
- GitHub Pages only supports **static content** (no backend/server code).

Practice Task:

1. Create a simple website or use your earlier projects.
2. Host it on GitHub Pages following the steps above.
3. Share the live link with your mentor or email: **workdinarventure@gmail.com**

📌 Submission Instructions:

- Send your **GitHub repository link** and **GitHub Pages live URL** to: **workdinarventure@gmail.com**
- Subject: `Day 13 - GitHub Pages Hosting`
- Write a short note about your experience and any challenges faced.
- Don’t copy instructions; explain in your own words what you did.

Day 14 & 15 – Final Project: Mini Portfolio Site

Objective:

Build a **mini portfolio website** showcasing your skills, projects, and contact information. This project will combine everything you've learned so far — HTML, CSS, JavaScript, and hosting.

Project Requirements

1. Homepage (index.html):

- Welcome message or your name and photo
- Short bio/intro about yourself
- Navigation menu (Home, About, Projects, Contact)

2. About Section:

- Your education, skills, and interests
- Use headings, paragraphs, and images

3. Projects Section:

- List 3 to 5 projects
- Each project with title, description, and a link/demo URL (if hosted)
- Use cards or boxes for each project

4. Contact Section:

- Contact form (Name, Email, Message)
- Social media links (GitHub, LinkedIn, Instagram, etc.)
- Use JavaScript to validate the form (basic validation)

Key Features to Implement

- Responsive design using CSS Flexbox/Grid & media queries (mobile-friendly)
- Navigation bar with smooth scrolling between sections
- Basic JavaScript for interactivity (e.g., form validation, button effects)
- Use of images/icons (you can use free icons from sites like FontAwesome)
- Clean, modern, and attractive UI/UX design

Suggested File Structure

```
/portfolio-site
  ├── index.html
  ├── css/
  │    └── styles.css
  ├── js/
  │    └── script.js
  └── images/
       └── your-photo.jpg
       └── project1.png
       └── project2.png
```

Step-by-Step Guide

Day 14:

- Setup project folder and files
- Create HTML structure: header, sections (Home, About, Projects, Contact)
- Add basic content and navigation menu
- Style layout with CSS (Flexbox/Grid)
- Make navigation links scroll smoothly

Day 15:

- Add images and project cards with descriptions
- Implement contact form with basic validation using JS
- Make website responsive (media queries for mobile/tablet)
- Test all links and functionality
- Upload to GitHub Pages (review Day 13 instructions)

Submission Instructions:

- Zip your entire project folder or share GitHub repo link
- Email to **workdinarventure@gmail.com**
- Subject: `Final Project - Mini Portfolio Site`
- Write a short description of your project and challenges you faced
- Make sure your portfolio site is live on GitHub Pages (share live URL)

💡 Tips to Impress:

- Use **consistent colors and fonts**
- Add hover effects on buttons and links
- Keep content clear and concise
- Make sure your site works on different screen sizes
- Add a professional photo or avatar